

جامعة حلب — كلية الهندسة المعلوماتية

قسم هندسة البرمجيات

نظام تحليل العقود التجارية الذكي v4.0 

100 سؤال وجواب للمناقشة الأكاديمية

مُعدّ لمناقشة مشروع التخرج — لجنة الحكم والإشراف

تمهيد

تغطي هذه الأسئلة جميع جوانب المشروع التي يُتوقع أن تُطرح أثناء المناقشة الأكاديمية: الفكرة العامة، الهيكلية البرمجية، الخوارزميات التقنية، قاعدة البيانات، مكتبات Python، نظام التفاوض، ودروس مستفادة. يُنصح الطالب بفهم كل جواب وصياغته بكلماته الخاصة.

القسم الأول: المقدمة والفكرة العامة

س1: ما هو هدف هذا المشروع بشكل مختصر؟

ج: تطوير نظام برمجي ذكي يُحلّل العقود التجارية العربية آلياً، يستخرج بنودها، يُقيّمها ويكشف البنود المفقودة، ويحدّر من خطأ التصنيف — كل ذلك دون اتصال بالإنترنت.

س2: لماذا اخترتم تحليل العقود التجارية تحديداً؟

ج: لأن مراجعة العقود تستهلك وقتاً كبيراً من المختصين القانونيين، والسوق العربي يفتقر لأدوات ذكية تُحلّل العقود بالعربية. المشروع يُقلّل التكاليف ويرفع دقة المراجعة الأولية.

س3: ما المشكلة التي يحلّها هذا النظام؟

ج: يُعالج ثلاث إشكاليات: (١) صعوبة استخراج النصوص العربية المعكوسة من PDF، (٢) ضعف المطابقة الحرفية بين أسماء البنود، (٣) غياب أداة مجانية ومستقلة لتقييم جودة العقود العربية.

س4: هل النظام يستبدل المحامي؟

ج: لا، النظام أداة مساعدة للمراجعة الأولية فقط. يُنبّه للبنود المفقودة ويُقيّم الهيكل، لكن التحليل القانوني المتعمق يبقى من اختصاص المحامين.

س5: ما لغة البرمجة المستخدمة ولماذا؟

ج: Python 3.11+ لأنها تمتلك بيئة مكتبات غنية لمعالجة النصوص (NLP) وتحليل PDF، وتدعم بناء واجهات رسومية بمكتبة CustomTkinter، وسهولة النشر والصيانة.

س6: ما إصدار النظام الحالي وما الجديد فيه؟

ج: الإصدار v4.0 يضيف: TF-IDF للمطابقة الدلالية، كاشف نوع العقد التلقائي، دعم OCR للصور الممسوحة، حفظ الإعدادات في JSON، وزر تكبير/تصغير الخط الديناميكي.

س7: كم نوعاً من العقود يدعم النظام؟

ج: ستة أنواع: العقد الدولي (10 بنود)، البند العام (8 بنود)، عقد الشراكة التجارية (10)، عقد الخدمات المهنية (10)، عقد الإيجار العقاري (10)، عقد العمل (10) — مجموع 58 بنوداً معيارياً.

س8: هل يعمل النظام بدون إنترنت؟

ج: نعم بالكامل. جميع الخوارزميات (TF-IDF، كاشف النوع، نظام النقاط) تعمل محلياً. الاتصال بالإنترنت اختياري فقط لـ Claude API إذا أراد المستخدم إضافة تحليل ذكاء اصطناعي.

س9: ما متطلبات تشغيل النظام؟

ج: Python 3.11+ مع مكتبات: scikit-learn، pdfplumber، customtkinter (إلزامية). اختياريًا: reportlab للتصدير، pillow+pytesseract للـ sentence-transformers، OCR للدقة العالية.

س10: ما حجم المشروع من حيث الكود؟

ج: 11 ملفاً رئيسياً (حوالي 5000 سطر كود) + مجلد negotiation يحتوي 4 ملفات. يستخدم قاعدة بيانات SQLite واحدة (contracts.db) بـ 5 جداول.

القسم الثاني: الهيكلية والمعمارية

س11: ما مبدأ التصميم المُتَّبَع في هيكلية المشروع؟

ج: مبدأ الفصل بين المخاوف (Separation of Concerns). كل ملف مسؤول عن طبقة واحدة: database.py للبيانات، auth.py للمصادقة، pdf_reader.py للاستخراج، clause_matcher.py للمطابقة... إلخ.

س12: كم طبقة في معمارية النظام؟

ج: أربع طبقات: طبقة العرض (main.py + negotiation/ui.py)، طبقة المنطق (pdf_reader)، طبقة الوصول للبيانات (database.py)، وطبقة التخزين (contracts.db).

س13: ما وظيفة ملف config.py؟

ج: يُدير الإعدادات الدائمة بين جلسات التشغيل عبر ملف settings.json. يحفظ: مفتاح API، حجم الخط، وضع المطابقة (tfidf/transformers)، وتفعيل OCR.

س14: ما وظيفة ملف font_manager.py؟

ج: يُدير حجم الخط ديناميكياً لكل التطبيق. يوفر FontManager الذي يُبَلِّغ جميع الـ Widgets المسجّلة عند تغيير الحجم، و ZoomBar الذي يوفر أزرار +/− في الشريط العلوي.

س15: لماذا فصلتم نظام التفاوض في مجلد منفصل؟

ج: لأن نظام التفاوض ميزة مستقلة ومتكاملة تمتلك قاعدة بياناتها وخوارزميتها وواجهتها. الفصل يُسهّل الصيانة، ويمكن إضافته أو إزالته دون التأثير على النظام الأساسي.

س16: ما تدفق البيانات من رفع الملف حتى عرض النتيجة؟

ج: رفع pdf_reader → PDF يستخرج النص ويصحح is_heading → RTL تُصَفّي الأسطر → clause_matcher يُطابق العناوين بـ analyze_rules → TF-IDF تحسب الدرجة → smart_mismatch_check_ تتحقق من التصنيف → عرض النتيجة وحفظها في SQLite.

س17: كيف تتواصل الملفات مع بعضها؟

ج: عبر استيرادات Python مباشرة (main.py). import يستورد من جميع الملفات. لا يوجد حافلة رسائل أو API داخلي — الاتصال مباشر ومتزامن ما عدا التحليل الذي يعمل في خيط منفصل (threading).

س18: لماذا استخدمتم threading في التحليل؟

ج: لأن استخراج PDF وتشغيل TF-IDF و OCR عمليات بطيئة. تشغيلها في الخيط الرئيسي يجمّد الواجهة. threading.Thread يُبقي الواجهة مستجيبة أثناء التحليل.

س19: ما نمط التصميم المُستخدم في الواجهة الرسومية؟

ج: نمط MVC بشكل مبسّط: Model = كائن State + قاعدة البيانات، View = كلاسات Controller، HomeView/AnalysisView/ReportView = دوال nav_ و on_type_ و run_ في كلاس .App

س20: كيف يتم الانتقال بين الشاشات؟

ج: دالة nav_() في App تُخفي جميع الشاشات بـ pack_forget() ثم تُظهر الشاشة المطلوبة بـ pack(). هذا النهج (Single Page) أسرع من إنشاء نوافذ جديدة في كل انتقال.

القسم الثالث: الخوارزميات الأساسية

س21: ما مشكلة Visual-RTL وكيف حللتوها؟

ج: بعض مولّدات PDF تُخزّن النص العربي بترتيب معكوس (Visual-RTL). حللناها بدالة `fix_line_()` التي: تكتشف السطر المعكوس، تعكس أحرفه، تُصلح رقم البند، وتُعيد عكس الأرقام المستقلة (كالسنوات). تعمل بدون مكتبات خارجية.

س22: ما معايير اكتشاف عناوين البنود؟

ج: أربعة معايير بالأولوية: (١) رقم + نقطة في البداية مثل '4. بند الدفع'، (٢) Article/Section/Clause في عقود إنجليزية، (٣) كلمة مفتاحية عربية من قائمة 45 كلمة + طول السطر > 65 حرفاً، (٤) أحرف إنجليزية كبيرة كليا (6-80 حرفاً).

س23: كيف يعمل نظام تقييم قوة العقد؟

ج: لكل بند موجود: 10+ لوجوده، 2+ إذا كان العنوان غامقاً، $1+3+5+8$ حسب عدد البنود الفرعية (4/3/2/1+). الدرجة النهائية = مجموع النقاط ÷ الحد الأقصى النظري $\times 100$.

س24: ما الحد الأقصى النظري لنظام النقاط؟

ج: عدد البنود المتوقعة $\times (10 + 2 + 8) =$ عدد البنود $\times 20$. مثلاً للعقد الدولي (10 بنود): الحد الأقصى = 200 نقطة. درجة $160/200 = 80\%$.

س25: ما مستويات تصنيف درجة قوة العقد؟

ج: خمسة مستويات: 80-100% (قوي جداً — أخضر)، 65-79% (جيد — سماوي)، 50-64% (متوسط — ذهبي)، 35-49% (ضعيف — برتقالي)، 0-34% (ضعيف جداً — أحمر).

س26: لماذا تحتاج المطابقة لخوارزمية ذكية وليس مطابقة حرفية؟

ج: لأن العقود الحقيقية تستخدم مصطلحات متنوعة: 'مدة الإيجار' قد تُكتب 'فترة التأجير' أو 'مدة السكن'. المطابقة الحرفية ستفشل في هذه الحالات، بينما TF-IDF يفهم التشابه الدلالي.

س27: كيف تعمل خوارزمية TF-IDF في المشروع؟

ج: نُوسّع النص بالمرادفات → نُحوّله لمصفوفة TF-IDF (بـ 2,4 `char_wb ngrams`) → نحسب cosine similarity بين عنوان العقد وكل بند معياري → نختار الأعلى تشابهاً بشرط تجاوز حد أدنى 0.12.

س28: لماذا استخدمتم `char_wb ngrams` بدلاً من `word ngrams`؟

ج: لأن char_wb (character-based n-grams) على حدود الكلمات أفضل للعربية التي تعتمد على الاشتقاق. كلمات 'السرية' و'سري' و'الإفصاح' ستشترك في n-grams مشتركة رغم اختلاف جذورها.

س29: ما قاموس المرادفات وكيف يعمل؟

ج: قاموس يحتوي +20 مجموعة مترادفات قانونية. عند المطابقة نُضيف مرادفات كل بند للنص قبل حساب TF-IDF. مثلاً 'شروط الدفع' تُوسّع بـ 'آلية السداد، طريقة الدفع، موعد الدفع، شروط السداد'.

س30: كيف يعمل كاشف نوع العقد التلقائي؟

ج: يُحلّل النص ويحسب نقاطاً لكل نوع بناءً على كلمات مفتاحية موزونة. مثلاً 'إيجار' وزنه 3.0 لصالح عقد الإيجار، 'تحكيم' وزنه 2.5 للعقد الدولي. النوع الأعلى نقاطاً يُعلن المكتشف مع نسبة ثقة.

القسم الرابع: قاعدة البيانات

س31: ما الفحص الذكي لتوافق نوع العقد؟

ج: مؤشران: (١) إذا كانت جميع البنود الموجودة = صفر رغم وجود عناوين → خطأ تصنيف واضح، يُشغّل كاشف النوع على نص العناوين فقط (بدون أسماء الشركات المُضَلَّلة). (٢) إذا كانت نسبة المطابقة > 20% مع +6 بنود → تحذير مع اقتراح النوع الصحيح.

س32: ما محرك قاعدة البيانات المستخدم ولماذا؟

ج: SQLite3 المدمج في Python. اخترناه لأنه: لا يحتاج خادماً منفصلاً، ملفه (contracts.db) يُنقل مع المشروع بسهولة، وأدأه كافٍ لتطبيق مكتبي ذي مستخدم واحد.

س33: كم جدولاً تحتوي قاعدة البيانات وما وظيفة كل منها؟

ج: خمسة جداول: (١) users — بيانات المستخدمين، (٢) analyses — نتائج تحليل العقود، (٣) neg_sessions — جلسات التفاوض، (٤) neg_clauses — بنود التفاوض، (٥) neg_history — سجل الإجراءات.

س34: كيف تُشفّر كلمات المرور؟

ج: بخوارزمية SHA-256 من مكتبة hashlib المدمجة. نُحوّل كلمة المرور النصية لـ bytes ثم نحسب تجزئتها. لا تُخزّن كلمة المرور الأصلية أبداً في قاعدة البيانات.

س35: ما العلاقات بين جداول قاعدة البيانات؟

ج: أربع علاقات واحد-إلى-كثير (1:users→analyses، 1:users→neg_sessions، neg_sessions→neg_clauses، neg_clauses→neg_history). جميعها مع ON DELETE CASCADE أي حذف المستخدم يحذف جميع بياناته.

س36: ما الفهارس الموجودة في قاعدة البيانات ولماذا؟

ج: ثلاثة فهارس مُركّبة: idx_analyses_user على (user_id, analyzed_at DESC) لتسريع استعلامات السجل، وفهرسان مشابهان لجدول neg_sessions و neg_clauses. بدونها تتباطأ الاستعلامات عند تراكم البيانات.

س37: كيف تُحفظ البيانات المركّبة (البنود الموجودة/المفقودة) في SQLite؟

ج: كقوائم JSON في حقل TEXT. مثلاً found_clauses = ["أطراف العقد", "مدة العقد"]. عند الاسترجاع نُحوّلها بـ json.loads(). هذا النهج أبسط من إنشاء جداول علاقات إضافية لمشروع أكاديمي.

س38: ما دور PRAGMA foreign_keys = ON في الكود؟

ج: يُفَعَّل التحقق من المفاتيح الخارجية في SQLite (معطّل افتراضياً) بدونه يمكن إدراج سجل في analyses بـ user_id غير موجود في users. هذا يضمن سلامة البيانات العلائقية.

س39: كيف تتعامل مع التزامن (Concurrency) في قاعدة البيانات؟

ج: النظام تطبيق مكتبي ذو مستخدم واحد في كل مرة، لذا التزامن ليس مشكلة فعلية. لكن استخدمنا PRAGMA journal_mode = WAL الذي يُحسّن الأداء ويسمح بقراءة متزامنة مع الكتابة.

س40: ماذا يحدث لبيانات المستخدم عند حذف حسابه؟

ج: بفضل ON DELETE CASCADE تُحذف جميع بياناته تلقائياً: تحليلاته في جدول analyses، وجلسات التفاوض في neg_sessions، وما يتفرع منها في neg_clauses و neg_history.

القسم الخامس: المكتبات والتقنيات

س41: كيف تتحقق من صحة بيانات التسجيل؟

ج: ثلاثة فحوصات: طول اسم المستخدم ≤ 3 أحرف، وجود @ في البريد الإلكتروني، طول كلمة المرور ≤ 6 أحرف. عند التكرار تُعيد (SQLite IntegrityError (UNIQUE constraint) تُترجمها لرسالة عربية واضحة.

س42: ما مكتبة الواجهة الرسومية ولماذا اخترتموها؟

ج: CustomTkinter — إصدار حديث من Tkinter يدعم التصميم الداكن والمكونات المعاصرة (أزرار مدوّرة، أشرطة تقدم، scrollable frames). اخترناها لأنها مجانية، مدمجة مع Python، ولا تحتاج تثبيت Qt أو wxPython الثقيل.

س43: ما مكتبة pdfplumber وكيف تستخدمونها؟

ج: مكتبة Python تستخرج النص والبيانات الوصفية من PDF. نستخدمها لاستخراج النص الخام من كل صفحة (extract_text)، واستخراج بيانات الكلمات مع fontname و size للكشف عن الخط الغامق (extract_words).

س44: ما مكتبة scikit-learn وكيف تستخدمونها؟

ج: مكتبة Python للتعلم الآلي. نستخدم منها: TfidfVectorizer لتحويل النصوص لمصفوفة TF-IDF، و cosine_similarity لحساب التشابه. استخدامنا لها بسيط ولا يشمل نماذج تعلم آلي معقدة.

س45: ما مكتبة pytesseract وعلاقتها ب Tesseract؟

ج: pytesseract هي wrapper بلغة Python لبرنامج Tesseract OCR الخارجي. تُحوّل الصورة لنص بتمثيل الصورة لـ Tesseract وإعادة النتيجة. Tesseract هو المحرك الفعلي ويجب تثبيته منفصلاً.

س46: ما sentence-transformers وكيف يختلف عن TF-IDF؟

ج: نموذج تعلم عميق يُحوّل الجمل لمتجهات في فضاء دلالي. يفهم المعنى الحقيقي — 'مدة الإيجار' و'فترة التأجير' ستحصلان على متجهات متقاربة. أدق من TF-IDF لكن يحتاج ~500MB وأبطأ عند أول استخدام.

س47: ما نموذج sentence-transformers المستخدم ولماذا؟

ج: paraphrase-multilingual-MiniLM-L12-v2 — يدعم +50 لغة بما فيها العربية، حجمه متوسط (~120MB)، ودقته جيدة للمهام القانونية. اخترناه لدعمه العربية والإنجليزية معاً وهو متوفر مجاناً على Hugging Face.

س48: كيف تتعاملون مع اختلاف أنظمة التشغيل في مسارات الملفات؟

ج: نستخدم `os.path.join()` و `os.path.abspath(__file)` لبناء المسارات بدلاً من الـ `hardcoded strings`. هذا يضمن عمل الكود على Windows (\) و Linux/Mac (/) دون تعديل.

س49: ما دور مكتبة `threading` وكيف استخدمتموها؟

ج: `threading.Thread` تُنشئ خيطاً منفصلاً للعمليات الثقيلة. استخدمنا `daemon=True` لإنهاء الخيط تلقائياً عند إغلاق البرنامج. نُعيد النتائج للواجهة بـ `self.after(0, callback)` لأن Tkinter ليس `thread-safe`.

س50: لماذا تستخدمون `self.after(0, callback)` بدلاً من استدعاء مباشر من الخيط؟

ج: لأن CustomTkinter/Tkinter ليس `thread-safe` — تعديل الـ `widgets` من خيط غير رئيسي قد يُسبب تعارضات وانهيأراً. `self.after(0, cb)` يُجدول تنفيذ `cb` في الخيط الرئيسي بأسرع فرصة.

القسم السادس: نظام التفاوض

س51: ما فكرة نظام التفاوض وكيف يعمل؟

ج: يُتيح لمدير الجلسة إدخال مواقف طرفي العقد على كل بند، ثم النظام يقترح صيغة مشتركة ذكية. يعتمد على Jaccard similarity لقياس الفجوة، ومنطق اقتراح متدرج حسب مستوى التشابه.

س52: ما حالات البند التفاوضي وما تسلسلها؟

ج: ست حالات: pending (بانتظار رد الطرف الثاني) → accepted/rejected/modified (رد الطرف الثاني) → suggested (اقتراح النظام) → agreed (إقرار نهائي). الحالة agreed تعني اكتمال التفاوض على هذا البند.

س53: كيف تعمل خوارزمية اقتراح الصيغة المشتركة؟

ج: ثلاثة مستويات: (١) تشابه $\leq 60\%$: نص الطرف الأول كأساس. (٢) تشابه $30-60\%$: دمج الموقفين بصيغة محايدة. (٣) تشابه $> 30\%$: الوسط الحسابي للأرقام + صيغة تفاوضية. مثال: 30 يوم + 60 يوم = 45 يوم مقترح.

س54: كيف تقيسون التشابه بين موقعي الطرفين؟

ج: Jaccard Similarity على مستوى الكلمات: $\frac{|\text{إتقاطع}|}{|\text{اتحاد}|}$ الكلمتين بعد حذف كلمات التوقف الشائعة. نتيجة 0.0 تعني لا تشابه، 1.0 تعني نفس النص تماماً.

س55: ما الفرق بين جلسة التفاوض المفتوحة والمغلقة؟

ج: المفتوحة (open): يمكن إضافة بنود وإجراء تعديلات. المغلقة (closed): تُغلق بقرار المدير عند الاتفاق أو انتهاء التفاوض. البنود المتفق عليها تبقى محفوظة ولا يمكن تعديلها.

س56: ما سجل الإجراءات (Audit Log) في نظام التفاوض؟

ج: جدول neg_history يُسجل كل إجراء: من قام به (actor: party_a/party_b/system)، نوع الإجراء (action: propose/accept/reject/modify/suggest/agree)، المحتوى، والتوقيت. يُتيح مراجعة تاريخ كامل لكل بند.

القسم السابع: OCR وتحليل الصور

س57: كيف يعمل نظام تحليل صورة العقد؟

ج: خمس مراحل: (١) المستخدم يرفع صورة PNG/JPG، (٢) ScannerCanvas يُشغّل Animation لخط ماسح أخضر، (٣) pytesseract يُطبّق OCR باللغتين ara+eng، (٤) is_heading () تستخرج العناوين، (٥) نفس analyze_rules () تُطبّق التحليل.

س58: كيف تُحسّنون الصورة قبل OCR؟

ج: ثلاث خطوات: تحويل للرمادي (Grayscale) لإزالة الألوان المُشوِّشة، مضاعفة التباين بـ ImageEnhance.Contrast(2.0)، تطبيق SHARPEN filter لتحديد حواف الأحرف. هذا يرفع دقة OCR خاصةً للنصوص الباهتة.

س59: ما معالم ScannerCanvas من الناحية التقنية؟

ج: Canvas مخصص يرسم خطأ أخضر (#00FF88) بسماكة 4 بكسل يتحرك بـ 2 بكسل/إطار عند FPS 60. التوهج (Glow) يُنشأ بـ 11 مستطيلاً بشفافية متدرجة. عند وصول الخط للنهاية يستدعي callback لبدء OCR.

س60: لماذا لا تعمل OCR على PDF مباشرة؟

ج: pdfplumber يستخرج النص الرقمي من PDF. OCR مطلوب فقط عندما يكون PDF عبارة عن صور ممسوحة ضوئياً (scanned). الكود يكتشف ذلك تلقائياً: إذا كان النص المستخرج > 20 حرف والصفحة تحتوي صوراً → يُطبّق OCR.

س61: ما دقة OCR للنصوص العربية؟

ج: تتراوح بين 75-90% حسب جودة الصورة. تُحسّن الدقة بـ: دقة الصورة ≤ 200 DPI، وضوح الطباعة، التباين الجيد بين النص والخلفية. الأخطاء الإملائية في OCR قد تُقلّل دقة اكتشاف البنود.

القسم الثامن: الواجهة والتجربة

س62: ما نظام إدارة الخطوط ولماذا صممتوه؟

ج: FontManager يُدير حجم الخط مركزياً لكل التطبيق. صمّمناه لأن تغيير حجم الخط في Tkinter يستلزم تحديث كل widget يدوياً. FM يستخدم نمط المراقب (Observer): الـ widgets المسجلة تُحدّث تلقائياً عبر قائمة listeners.

س63: ما حدود التكبير/التصغير للخط؟

ج: من 0.7× (أصغر حجم) إلى 1.8× (أكبر حجم) من الحجم الأساسي. الحجم الافتراضي 1.0×. يُحفظ في settings.json ويُستعاد عند إعادة التشغيل. الخط الأساسي لـ body مثلاً 15pt، عند 1.8× يصبح 27pt.

س64: كيف نتحقق من صحة ملف PDF المرفوع؟

ج: أربعة فحوصات في pdf_reader: (١) وجود الملف في المسار، (٢) الامتداد pdf، (٣) الحجم > 100MB، (٤) محاولة فتحه بـ pdfplumber مع التقاط استثناءات محددة (كلمة مرور، ملف تالف، لا صلاحية).

س65: كيف تعرضون رسائل الخطأ للمستخدم؟

ج: نُترجم جميع الاستثناءات التقنية لرسائل عربية مفهومة في pdf_reader. مثلاً 'password' في الاستثناء → 'الملف محمي بكلمة مرور'. نعرضها بـ messagebox.showerror() ونُحدّث شريط الحالة.

س66: ما شريط الحالة في الأسفل؟

ج: شريط بارتفاع 30px في أسفل النافذة يعرض الحالة الحالية للمستخدم. يُحدّث في كل انتقال ويعرض: 'الشاشة الرئيسية'، 'التحليل — نوع العقد'، 'تقرير التفصيلي! ... إلخ. يعرض أيضاً تحذيرات المكتبات غير المثبتة.

س67: كيف تُعالجون التحقق من نوع العقد قبل التحليل؟

ج: فحصان في run_(): (١) إذا لم يختَر المستخدم نوع العقد → رسالة تحذير ومنع التحليل. (٢) إذا لم يُرفع ملف → رسالة تحذير. هذا يمنع تشغيل التحليل على حالة فارغة.

القسم التاسع: الأمان والجودة

س68: ما تدابير الأمان المُطبَّقة في النظام؟

ج: أربعة تدابير: (١) تشفير كلمات المرور بـ SHA-256، (٢) لا تُخزَّن كلمة المرور الأصلية أبداً، (٣) COLLATE NOCASE في SQL لمنع حالة الأحرف في اسم المستخدم، (٤) ON DELETE CASCADE لتنظيف البيانات المتعلقة عند الحذف.

س69: هل النظام عرضة لـ SQL Injection؟

ج: لا. نستخدم Parameterized Queries (العلامة ؟) في جميع استعلامات SQLite. مثلاً:
`((cursor.execute('SELECT * FROM users WHERE username=?', (username`
 هذا يمنع حقن SQL تلقائياً.

س70: كيف تضمنون جودة الكود (Code Quality)؟

ج: عدة ممارسات: تسمية واضحة للمتغيرات والدوال بالعربية والإنجليزية، Docstrings لكل دالة رئيسية، فصل المخاوف (ملف = مسؤولية واحدة)، لا بيانات وهمية (demo data) عند الفشل، رسائل خطأ واضحة بدلاً من إخفائها.

س71: ما سياسة التعامل مع الاستثناءات (Exceptions)؟

ج: ثلاثة مستويات: (١) استثناءات PDF تُرفع كـ ValueError برسائل عربية وتُعرض للمستخدم. (٢) استثناءات قاعدة البيانات تُلتقط وتُطبع في الكونسول كتحذيرات. (٣) استثناءات Listeners تُلتقط لمنع انهيار دالة notify.

س72: هل هناك اختبارات وحدة (Unit Tests)؟

ج: لا اختبارات رسمية بـ pytest، لكن نَقَدنا اختبارات يدوية موقَّعة: اختبار دقة TF-IDF على 4 عناوين وتحقق من صحة المطابقة، واختبار كاشف النوع على نصِّي إيجار ودولي مع قياس نسبة الثقة.

القسم العاشر: المقارنات والتحديات

س73: ما التحدي التقني الأصعب في المشروع؟

ج: معالجة النص العربي المعكوس (Visual-RTL) من ملفات PDF. لا توجد مكتبة جاهزة تحل هذه المشكلة بدقة للعقود العربية. طوّرتنا خوارزمية مخصصة بأربع خطوات تعمل بدون مكتبات خارجية.

س74: ما قيود النظام الحالية؟

ج: ثلاثة قيود: (١) يحلّ هيكل العقد لا معناه القانوني العميق. (٢) دقة OCR تعتمد على جودة الصورة الممسوحة. (٣) قاموس المرادفات محدود — مصطلحات غير موجودة فيه قد لا تُطابق بدقة.

س75: كيف يختلف مشروعكم عن ChatGPT في تحليل العقود؟

ج: مشروعنا: مجاني، يعمل بدون إنترنت، متخصص بالعقود العربية المحددة، مُفسّر (يشرح لماذا البند موجود أو مفقود). ChatGPT: عام، يحتاج إنترنت وحساباً، تحليله غير موثوق 100% ولا يُعطي درجة منظمة.

س76: ما الفرق بين TF-IDF وword embeddings؟

ج: TF-IDF: يمثّل النص كمصفوفة أعداد بناءً على تكرار الكلمات، سريع ولا يحتاج تدريباً. Word Embeddings (كـ sentence-transformers): يُحوّل النص لمتجه يعكس المعنى الدلالي، أدق لكن أبطأ ويحتاج نموذجاً مُحمّلاً.

س77: لماذا اخترتم SQLite بدلاً من MySQL أو PostgreSQL؟

ج: لأن التطبيق مكتبي ذو مستخدم واحد. MySQL/PostgreSQL تحتاج خادماً منفصلاً وإعداداً معقداً. SQLite ملف واحد يُنقل مع البرنامج، مدمج في Python، وأداؤه أكثر من كافٍ لهذا الاستخدام.

س78: ما مدى قابلية النظام للتوسع (Scalability)؟

ج: التصميم يدعم التوسع بسهولة: إضافة نوع عقد جديد يستلزم إضافة قاموس في CONTRACT_TYPES فقط. إضافة بنود مرادفة تعديل SYNONYMS_ في clause_matcher. للتوسع لعدة مستخدمين يمكن استبدال SQLite بـ PostgreSQL.

القسم الحادي عشر: أسئلة تقنية متقدمة

س79: ما Cosine Similarity وكيف تُستخدم هنا؟

ج: مقياس يحسب زاوية بين متجهين. قيمة 1.0 تعني المتجهان في نفس الاتجاه (متطابقان تماماً)، 0.0 تعني متعامدان (لا علاقة). نستخدمه لقياس تشابه عنوان البند مع البنود المعيارية في فضاء TF-IDF.

س80: ما N-grams وكيف تُطبق على العربية؟

ج: N-gram هو تسلسل متجاور من N عناصر. char_wb ngrams(2,4) تعني: خذ كل 2 و 3 و 4 أحرف متجاورة من كل كلمة. مثلاً كلمة 'العقد': bigrams ['ال', 'الع', 'عق', 'قد'], trigrams ['الع', 'لقد', 'عقد', 'قد... إلخ. هذا يُقاوم الأخطاء الإملائية.

س81: ما Observer Pattern وكيف يُطبق في FontManager؟

ج: نمط تصميم حيث Subject (FontManager) يحتفظ بقائمة Observers (listeners). عند تغيير الحجم يستدعي notify () التي تُنفذ كل listener. كل widget مسجلة ك listener كـ ('body' font=FM.font('body' listener: lambda: label.configure(font=FM.font('body

س82: ما Singleton Pattern وهل تستخدمونه؟

ج: نمط يضمن وجود نسخة واحدة من الكائن. نستخدمه ضمناً: FM = FontManager () و ST = State () (تُعرف مرة واحدة على مستوى الموديول كمتغيرات عامة. أي كود يستوردها يحصل على نفس النسخة.

س83: ما الفرق بين pack () و grid () و place () في Tkinter؟

ج: pack (): يُرتب الـ widgets بشكل خطي (أفقي/عمودي)، الأسهل للتخطيطات البسيطة. grid (): يُرتبها في شبكة صفوف وأعمدة، مناسب للنماذج. place (): تحديد موضع مطلق، نادر الاستخدام. استخدمنا pack () بشكل رئيسي لبساطته.

س84: كيف تعملون مع RTL (Right-to-Left) في CustomTkinter؟

ج: Tkinter لا يدعم RTL أصلياً بشكل كامل. نتحايل بـ: (١) استخدام anchor='e' للنصوص العربية، (٢) justify='right'، (٣) ترتيب الـ widgets بعكس الاتجاه (side='right' قبل side='left')، (٤) تصحيح النص في pdf_reader قبل العرض.

س85: ما الفرق بين lambda و def في Python وأين تستخدمون كلاً منهما؟

ج: lambda: دالة مجهولة من سطر واحد تُستخدم عند استدعائها مرة واحدة. def: دالة مسمّاة لها جسم. نستخدم lambda في أوامر الأزرار (command=lambda: self._nav('home')) وفي قوائم listeners. ندفع دوال تبلغ +3 أسطر في def.

س86: ما معنى daemon=True في threading.Thread؟

ج: يُحوّل الخيط لـ daemon thread أي يُنهي تلقائياً عند انتهاء البرنامج. بدونهُ قد يبقى الخيط يعمل في الخلفية بعد إغلاق النافذة ويمنع إنهاء العملية. لازمٌ للخيط التي تُشغل عمليات طويلة مثل التحليل.

القسم الثاني عشر: التطوير والمستقبل

س87: ما الميزات التي يمكن إضافتها في الإصدار القادم؟

ج: خمس ميزات مقترحة: (١) دعم DOCX بجوار PDF، (٢) نموذج تعلم آلي مدرب على عقود عربية حقيقية، (٣) تحليل محتوى البنود لا فقط وجودها، (٤) واجهة ويب بـ Flask/FastAPI، (٥) تصدير تقرير كامل بـ Arabic PDF مع خط عربي صحيح.

س88: ما التحسينات الممكنة على خوارزمية المطابقة؟

ج: ثلاثة تحسينات: (١) بناء نموذج BERT مدرب على عقود عربية (Arabic Legal BERT). (٢) توسيع قاموس المرادفات بالتشاور مع محامين. (٣) استخدام fuzzy matching لمعالجة الأخطاء الإملائية في OCR.

س89: كيف يمكن نشر النظام (Deployment) خارج البيئة المحلية؟

ج: خيارات: (١) تحويل لـ exe بـ PyInstaller للتوزيع على Windows بدون Python. (٢) تحويل الواجهة لـ Web UI بـ Flask + HTML/CSS. (٣) نشر كـ Web Service على السحابة مع PostgreSQL بدلاً من SQLite.

س90: لماذا لم تستخدموا REST API لربط الواجهة بالمنطق؟

ج: لأن التطبيق مكتبي أحادي العملية ولا حاجة لبنية REST API. Client-Server يُضيف تعقيداً غير مبرر (خادم HTTP، تسلسل JSON، طلبات شبكة). الاستدعاء المباشر في Python أسرع وأبسط لهذا النوع من التطبيقات.

القسم الثالث عشر: أسئلة ختامية وتقييمية

س91: ما أكثر شيء تعلّمتموه من هذا المشروع؟

ج: معالجة النصوص العربية في PDF — مشكلة معقدة لا توثيق كافٍ لها. وكيفية تصميم نظام متكامل بطبقات منفصلة قابلة للصيانة. وكيف تختلف النظرية (معالجة اللغة الطبيعية) عن التطبيق الفعلي على بيانات حقيقية.

س92: كيف تقيسون نجاح النظام؟

ج: بثلاثة مقاييس: (١) دقة استخراج العناوين < 89% على ملفات PDF حقيقية. (٢) صحة المطابقة: TF-IDF يطابق 'مدة الاتفاقية' بـ 'مدة العقد' و'آلية السداد' بـ 'شروط الدفع'. (٣) وقت التحليل > 3 ثوانٍ لملفات 10 صفحات.

س93: ما نتائج اختبار النظام على عقود حقيقية؟

ج: اختبرنا 4 ملفات: عقد تعاون دولي (صفحتان) — دقة استخراج 100%، عقد خدمات — 91.7%، عقد شراكة — 88.9%، عقد إيجار — 92.9%. متوسط الدقة 93.4% وهو أعلى من هدفنا 85%.

س94: لماذا يُعدّ هذا المشروع ذا قيمة أكاديمية؟

ج: يُقدّم إسهامات في ثلاثة مجالات: (١) خوارزمية جديدة لتصحيح Visual-RTL الموثقة لأول مرة باللغة العربية. (٢) تطبيق عملي لـ TF-IDF في المجال القانوني العربي. (٣) نظام متكامل يربط NLP بواجهة رسومية وقاعدة بيانات.

س95: ماذا تقول لجنة التحكيم عن حدود المشروع بصدق؟

ج: النظام يُحلّل هيكل العقد ومدى اكتماله، لا يُحلّل المحتوى القانوني التفصيلي. لا يستطيع كشف البنود التعسفية أو غير القانونية. دقة OCR تعتمد على جودة الصورة. هذه حدود معلنة نذكرها في الأطروحة بوضوح.

س96: ما الفرق بين هذا المشروع وأنظمة تحليل العقود الموجودة تجارياً؟

ج: الأنظمة التجارية (مثل Kira، Luminance) تعمل بالإنجليزية أساساً وتكلف آلاف الدولارات. مشروعنا: مجاني، عربي كامل، يعمل دون إنترنت، مفتوح المصدر، ويمكن تخصيصه بسهولة لأنواع عقود جديدة.

س97: ما رأيكم في استخدام الذكاء الاصطناعي التوليدي (GPT) في هذا المجال؟

ج: له مزايا في فهم السياق القانوني العميق، لكن عيوبه للمشاريع الأكاديمية: يحتاج إنترنت واشتراكاً مدفوعاً، نتائج غير حتمية (نفس السؤال قد يعطي إجابات مختلفة)، ولا يعطي درجة منظمة قابلة للقياس والمقارنة.

س98: كيف يمكن توسيع النظام لدعم عقود بلغات أخرى؟

ج: أربعة تعديلات: (١) إضافة HEADING_KEYWORDS للغة الجديدة. (٢) إضافة قاموس أنواع عقود جديد في CONTRACT_TYPES. (٣) تدريب TF-IDF على نصوص اللغة الجديدة. (٤) استبدال نموذج sentence-transformers بنموذج متعدد اللغات (مثل المستخدم حالياً).

س99: ما خلاصة ما حققه هذا المشروع؟

ج: نجح المشروع في بناء نظام مكتبي ذكي متكامل لتحليل العقود التجارية العربية، يعمل بالكامل دون إنترنت، بدقة استخراج تجاوزت 89%، مع واجهة رسومية احترافية ونظام تفاوض وإمكانية تحليل الصور — كل ذلك مجاناً وبكود مفتوح وقابل للتوسع.

س100: ما توصيتكم للطلاب الذين يريدون تطوير مشاريع مشابهة؟

ج: أربع توصيات: (١) ابدأ بالمشكلة الحقيقية لا بالتقنية — اجمع عقوداً حقيقية وافهم نقاط الضعف فيها. (٢) استخدم مبدأ الفصل بين المخاوف منذ البداية — يُوقَّر الكثير عند التعديل. (٣) اختبر على بيانات حقيقية مبكراً — النتائج المختبرية تختلف كثيراً عن الواقع. (٤) وثِّق قراراتك التصميمية — لجنة التحكيم تُقَدِّر 'لماذا' أكثر من 'ماذا'.

————— نهاية الأسئلة | بالتوفيق في المناقشة —————